

SESSION 2

Operators, Expressions & Type Casting

How C evaluates expressions — and how to control that evaluation

Types of Operators

An operator tells the compiler to perform a specific operation on operands.



Arithmetic

`+` `-` `*` `/` `%`

Basic math: addition, subtraction, multiplication, division, modulus



Relational

`<` `>` `<=` `>=` `==` `!=`

Compares two values → returns true(1) or false(0)



Logical

`&&` `||` `!`

Combines or reverses conditions



Assignment

`=` `+=` `-=` `*=` `/=`

Assigns or updates a variable's value



Increment / Decrement

`++` `--`

Increases or decreases a value by 1

Operator Precedence

When an expression has more than one operator, precedence decides which one is evaluated first.

Precedence	Operators	Description
Highest	()	Parentheses — evaluated first, always
	* / %	Multiplication, Division, Modulus
	+ -	Addition, Subtraction
	< > <= >=	Relational comparisons
	== !=	Equality checks
	&&	Logical AND / OR
Lowest	= += -= ...	Assignment — evaluated last

Example: `a + b * 2` → `*` runs first (higher precedence) → `a + (b * 2)`

Associativity

When two operators share the SAME precedence, associativity decides the direction of evaluation.

Most arithmetic operators are **left-to-right** associative — evaluation moves from the leftmost operator toward the right.

Walkthrough Example

```
result = 20 / 4 * 2;
```

Step 1

$20 / 4 \rightarrow 5$ (left operator evaluated first)

Step 2

$5 * 2 \rightarrow 10$ (moving left to right)

Explain it as:

“When two operators of the same rank stand side by side, C reads them like a sentence — left to right — solving one and then moving to the next.”

Note: Assignment (=) and unary operators are right-to-left associative instead.

Expressions vs Instructions

Expression:

a combination of variables, constants and operators that evaluates to a single value.

Instruction:

an expression terminated with a semicolon — a complete statement the compiler executes.

```
result = a + b * 2 - 1;
```

expression

*The whole line — including the semicolon — is the instruction. The part “a + b * 2 - 1” is the expression inside it.*

Type Casting — Overview

Converting a value from one data type to another so it behaves correctly in an expression.

Why it matters: C is a strongly-typed language, so mixing data types in one expression (e.g. an int with a float) requires the values to be brought to a common type before the operation can happen. This conversion happens in one of two ways:

IMPLICIT CASTING

Automatic

Done automatically by the compiler, following C's built-in type promotion rules. No action needed from the programmer.

Example: `int + float → float`

EXPLICIT CASTING

Manual

Performed deliberately by the programmer using the cast operator (`data_type`), to force a specific, controlled conversion.

Example: `(float)a / b`

Implicit Type Casting (Automatic)

The compiler promotes the “smaller” data type to the “larger” one before evaluating: char → int → float → double.

Situation	Code Example	What Happens	Result
int with float	<pre>int a=5; float b=2.5; result = a + b;</pre>	a is promoted to float (5.0) before addition	7.50
char in arithmetic	<pre>char c='A'; int x = c + 1;</pre>	c is promoted to int using its ASCII value (65)	66
int with double	<pre>int p=7; double q=2.0; result = p / q;</pre>	p is promoted to double before division	3.5
assigning smaller to larger	<pre>float f = 10;</pre>	The integer 10 is widened to 10.0 automatically	10.00

Key idea: implicit casting always widens (moves to a type that can safely hold more information) so no data is lost.

Explicit Type Casting (Manual)

Written as (data_type)expression — the programmer forces a conversion instead of waiting for the compiler.

Situation	Code Example	What Happens	Result
avoid integer division	<code>int a=10, b=3; (float)a / b</code>	a is cast to float BEFORE the division, so the fraction is kept	3.33
truncating a decimal	<code>float x = 9.7; (int)x</code>	The decimal part is dropped (not rounded)	9
character to ASCII code	<code>char ch = 'A'; (int)ch</code>	Converts the character to its numeric ASCII value	65
number back to character	<code>(char)66</code>	Converts the ASCII value back into its character	'B'
accurate average	<code>int sum=17, count=5; (double)sum / count</code>	sum is cast to double so the average keeps decimals	3.4

Key idea: explicit casting can narrow a value (lose precision or truncate) — it is a deliberate, controlled trade-off the programmer chooses to make.

math.h Library Functions

Predefined functions that simplify common mathematical calculations. Most return a double.

*Include with: **#include <math.h>***

Function	Purpose	Example	Result
sqrt(x)	Square root of x	sqrt(9)	3.00
pow(x, y)	x raised to the power y	pow(2, 3)	8.00
abs(x)	Absolute (positive) value of x	abs(-5)	5
ceil(x)	Rounds x up to the next whole number	ceil(4.2)	5.00
floor(x)	Rounds x down to the previous whole number	floor(4.8)	4.00